Agent Mesh - User Stories

Team 2 - Phoenix. Sprints 1 to 3, plus application test cases for Sprint 1.

Sprint	Goal at end of sprint	Stories	Points
Sprint 1	an agent can be enrolled through a defined workflow, the system spawns the right agents from a user's brief, each agent's data is stored in dynamically created, schema-validated tables, and users can do all of this from the frontend.	**5**	**42**
Sprint 2	the agents spawned in Sprint 1 are connected into a workflow graph that actually runs, with sequential and parallel steps, retries, live progress and real specialist agents doing the work.	**3**	**39**
Sprint 3	every step is routed to a suitable model with automatic fallback, the user can see what each workflow cost, and the output is delivered as one synthesised result in a chat thread.	**3**	**31**
Total		**11**	**112**

Story points use a Fibonacci-style scale (1, 2, 3, 5, 8, 13). Larger numbers mean more effort, complexity or risk.

Sprint 1 - Agent Enrollment, Agent Spawning and Frontend

End of sprint: an agent can be enrolled through a defined workflow, the system spawns the right agents from a user's brief, each agent's data is stored in dynamically created, schema-validated tables, and users can do all of this from the frontend.

US_ID	Feature	User Story	Acceptance Criteria	Story Points
US_01	**Agent Enrollment**	As a platform engineer, I want to enroll a new agent through one defined workflow so that the orchestrator can discover and use it without code changes.	- Enrollment takes name, description, capabilities, supported models, input/output schema and cost profile. - An enrolled agent appears in the registry with a unique ID and an Active status. - Missing required fields and duplicate agent names are rejected with a clear error. - Enrollment completes in a single request and in under 3 seconds. - An agent can be updated or disabled; disabled agents are never offered to the orchestrator.	**8**
US_02	**Agent Spawning**	As a user, I want the system to read my brief and spawn the right agents, more than one when needed, so that complex requests are handled by a coordinated team.	- The system maps the brief to required capabilities and spawns one or more enrolled agents. - Only the agents needed for the brief are spawned; the rest are skipped with a reason. - Each spawned agent gets its own instance ID and status. - If no enrolled agent matches, nothing is spawned and the user is told which capability is missing. - Empty or vague briefs are rejected or answered with a clarifying question.	**13**
US_03	**Dynamic Table Creation**	As a data analyst, I want a table created automatically for each agent from its output schema so that structured results are stored without manual database changes.	- A table is created when an agent is enrolled, with columns and types taken from its output schema. - Table names are generated safely and unique; unsafe names or SQL in names are rejected or sanitised. - Running creation twice does not produce an error or a duplicate table. - If table creation fails, enrollment is rolled back and no half-created agent remains.	**8**
US_04	**Schema Validation**	As a user, I want agent schemas and outputs validated so that bad data never reaches storage or the next step.	- A malformed schema is rejected at enrollment with an error naming the problem field. - Agent output is validated against the schema before it is written to the table. - Output with a missing field or wrong data type is rejected, the step is marked Failed and nothing is stored. - Validation errors are logged with agent ID and field.	**5**
US_05	**Frontend**	As a user, I want a workspace where I can submit a brief, see enrolled agents and watch spawned agents so that I can use the platform without calling the API.	- The /office page has a brief input box and submit button; empty briefs are blocked. - An agents page lists enrolled agents with capabilities and status, matching the API. - After a brief is submitted, spawned agents appear with their status and selection reason. - Validation and error messages from the backend are shown clearly to the user.	**8**

Sprint 2 - Workflow Engine and Agent Execution

End of sprint: the agents spawned in Sprint 1 are connected into a workflow graph that actually runs, with sequential and parallel steps, retries, live progress and real specialist agents doing the work.

US_ID	Feature	User Story	Acceptance Criteria	Story Points
US_06	**Workflow Planning**	As a user, I want my spawned agents arranged into a workflow plan so that they run in the right order with the right dependencies.	- A workflow DAG is generated with agents as nodes and dependencies as edges. - The plan supports sequential steps, parallel branches with joins, and conditional steps. - Circular dependencies are detected and rejected before execution. - The plan is saved and can be viewed in the workspace.	**13**
US_07	**Workflow Execution**	As a user, I want the workflow to run on its own with live progress so that complex jobs finish without manual prompting and I can see where they stand.	- Independent steps run in parallel; dependent steps wait for their inputs. - Output from one agent is validated, stored in its table and passed to the next agent. - A failed step is retried up to 3 times with backoff, then marked Failed with a reason; completed results are kept. - Each step shows Pending, Running, Completed, Failed or Skipped and updates without a page refresh.	**13**
US_08	**Specialist Agents**	As a software engineering lead or data analyst, I want working specialist agents so that I can run a review-fix-test or clean-analyse-report pipeline from one request.	- Each agent has an input/output schema, a table and unit tests. - The code auditor returns issues by severity; the code fixer acts only on those issues; the test writer produces runnable tests. - The data cleaner returns a cleaned dataset with a change summary; the analyst and report writer turn it into a readable report.	**13**

Sprint 3 - Multi-Provider Routing, Cost Transparency and Final Result

End of sprint: every step is routed to a suitable model with automatic fallback, the user can see what each workflow cost, and the output is delivered as one synthesised result in a chat thread.

US_ID	Feature	User Story	Acceptance Criteria	Story Points
US_09	**Model Routing**	As a user, I want each agent step routed to a suitable model across providers, with automatic fallback, so that workflows complete reliably without wasting spend.	- Adapters for OpenAI, Anthropic and Google Gemini sit behind one shared interface. - Routing considers task type, cost and latency, and the model used for each step is stored and shown. - If a provider errors or is rate-limited, the step switches to the fallback model within 5 seconds. - The user can see when a fallback was used.	**13**
US_10	**Cost Transparency**	As a technical product manager, I want cost, token usage and latency per agent and per workflow so that I can audit spend and performance.	- Tokens, estimated cost and latency are recorded for every step. - A workflow summary shows totals and a per-agent breakdown. - Cost data is saved so it can be reviewed later.	**5**
US_11	**Result Synthesis**	As a user, I want all validated agent outputs combined into one clear result in a chat thread so that I can read it and ask follow-ups.	- The final result appears in the chat thread and credits the agent behind each part. - The user can send a follow-up in the same thread and context is kept. - The result and conversation are saved and reload when the thread is reopened.	**13**